

Unit – 1 : Session – 12 : SLO – 1 | Dynamic Dispatch & Overhead

Activity (Solutions)

- 1) `class Base { void show(){ System.out.println("Base"); } }` `class Derived extends Base { @Override void show(){ System.out.println("Derived"); } }` `Base b = new Derived(); b.show();` → prints Derived (dynamic dispatch via base reference).
- 2) Virtual method table (vtable): each class with overridable methods has a table mapping method slots to implementations; an object stores a pointer to its class's vtable. For `Base b = new Derived(); b.show();` the JVM reads Derived's vtable entry for `show()` and jumps to `Derived.show` — this is how overriding is resolved at run time.
- 3) Direct vs virtual call: a direct (static/private/final) call binds at compile time — the address is fixed, so it is faster and inlinable. A virtual call must load the vtable pointer and index the table at run time — a small overhead (extra indirection, harder to inline), in exchange for polymorphic flexibility.